# Exploring DeepFakes: Face Reenactment Report

October 12, 2026

## 1 Rectified Flow Head Swap

### 1.1 Scope and hardware

We wanted to swap a video face with one from a selfie. The run had to stay on one laptop with a 4 GB NVIDIA GPU and 8 GB of system RAM. We sent no data to a cloud service.

The output had to keep the source motion, expression, lighting and background. Only the head should change. The clip is 96 frames at 512x512 and lasts 3.2 seconds.

### 1.2 How we measured

Every number comes from one script run on the same 96 frames, and the results table reports it. Face change is how much the face region moves from one frame to the next. The unswapped footage sets the level to match, so a value far below it means the smoother removed real motion too. Consecutive identity is how similar a face is to the one before it, measured in a face recognition embedding, so higher is more stable. Identity to the portrait measures similarity to the reference portrait; the unswapped footage scores near zero because it is a different person. Hair noise is how much the hair region flickers, and lower is better; p95 marks the worst frames, the value 95 percent stay under.

### 1.3 Pipeline overview

The flow has four stages. Generation transfers the identity, restoration cleans each face with GFPGAN, smoothing removes the boil, and reassembly attaches the audio. Only the last stage writes a video file.

The boil is the flicker that comes from generating each frame on its own. Figure 1 shows the chain in order, with the cost of each stage.

### 1.4 How a rectified flow model works

A rectified flow model learns one job: connect a clean image and noise with a straight line. Training picks one random time per example and mixes the two ends at that time. The model predicts the direction that points from the noise end to the image end. The guess is compared with the true direction along the line, and the weights are then nudged.

At sampling time the same model starts from pure noise. Each Euler step moves the sample a small way along the predicted direction, and an image forms. The sampling code calls these steps denoising steps, because the sample becomes less noisy at each step. Figure 2 draws training and sampling side by side.

We did not train the base model; we used a released checkpoint. The released model is step distilled, so four steps are enough for it. We could not tell the difference between four steps and eight, and eight took twice the time.

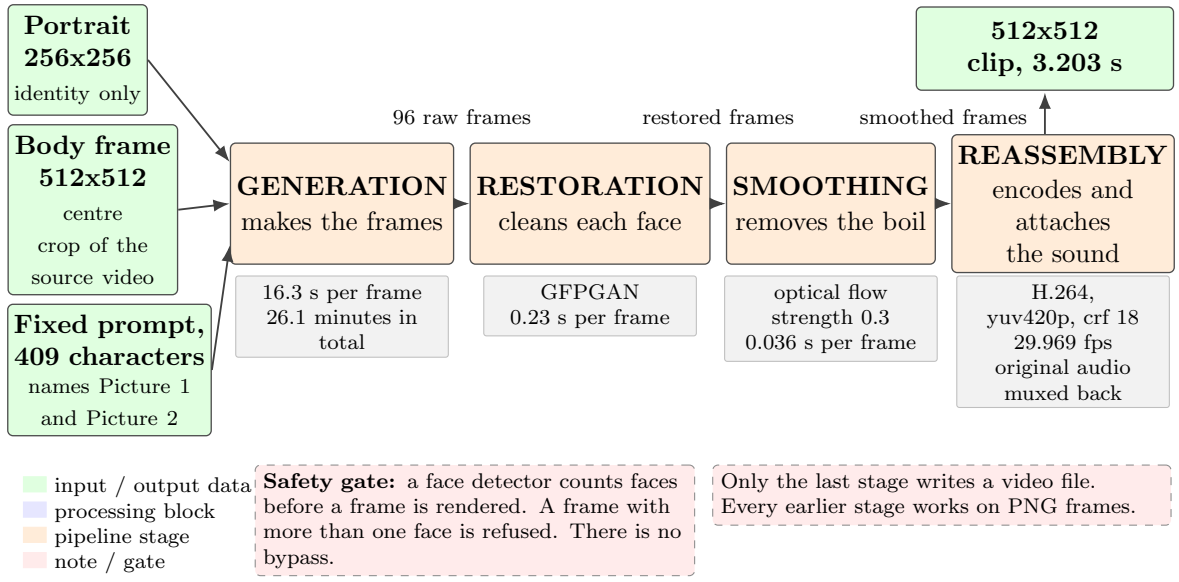



Figure 1: The four-stage pipeline: generation, restoration, smoothing and reassembly, with the cost measured for each stage.

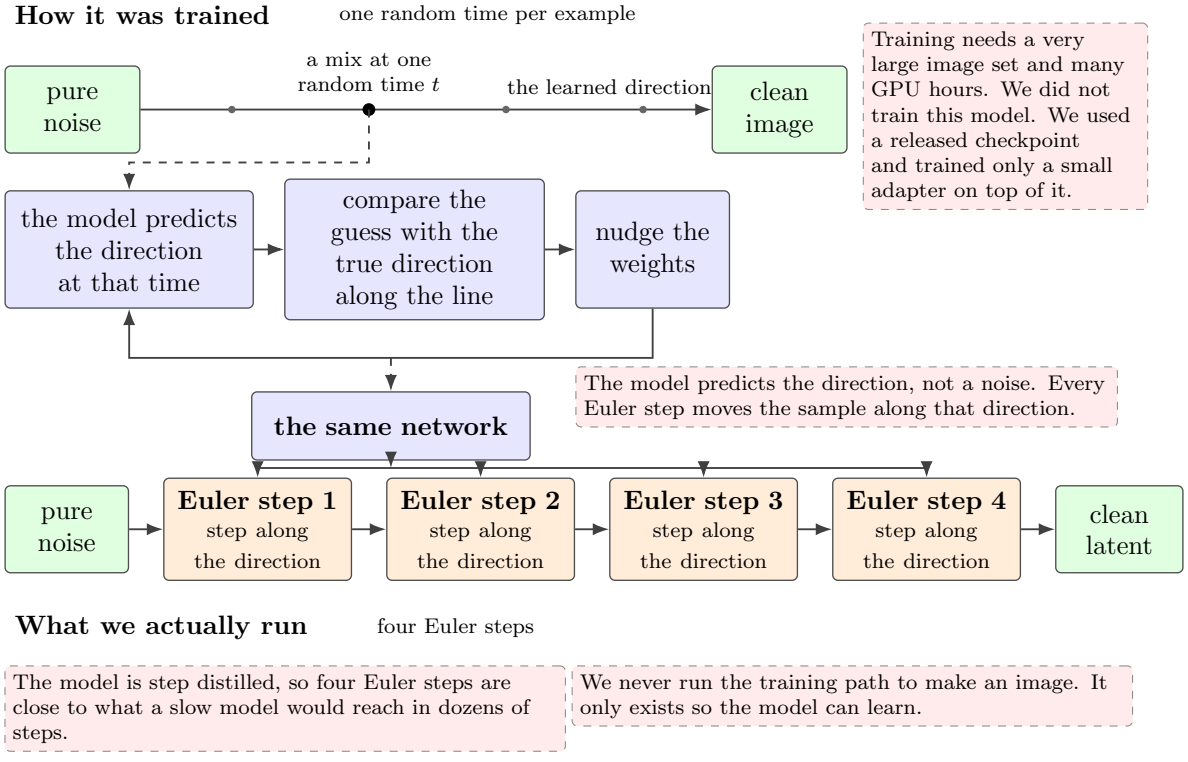



Figure 2: The rectified flow model in training and at sampling. Training draws one random time per example, mixes the image and noise at that time, and teaches the model the direction along the straight line between the two ends. Sampling starts from noise and takes four Euler steps along the predicted direction. Both jobs use one and the same network.

## 1.5 The model and the LoRA adapter

The model is a stack of transformer blocks (Figure 3). A latent token of 128 channels is projected to width 3072. It passes through five double-stream blocks, then 20 single-stream blocks and a final layer. In the first part the text and image tokens stay apart, and they meet through joint

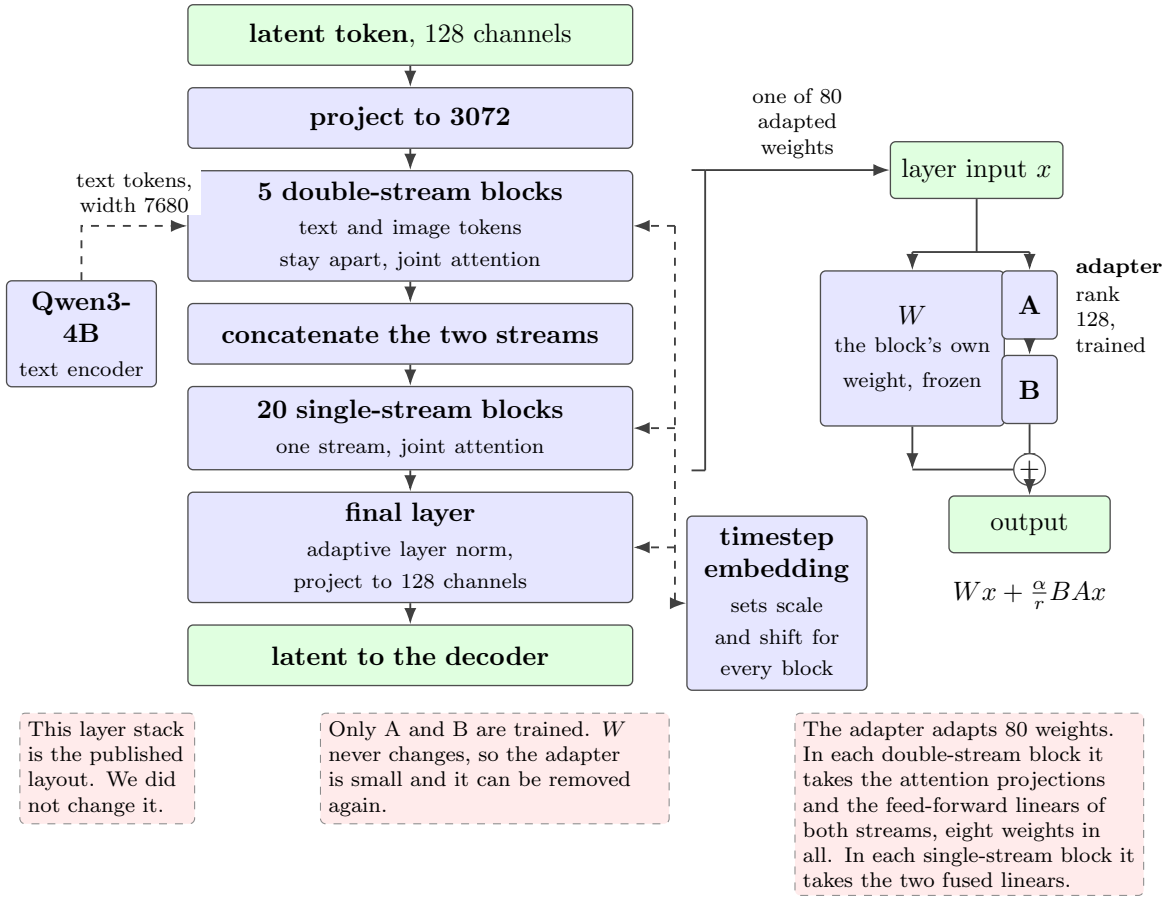



Figure 3: The model path: the latent token is projected to width 3072, passes the double-stream and single-stream blocks, and the final layer returns the latent for the decoder. The text encoder and the timestep embedding enter as conditioning. The bracket on the right marks one weight inside a block, where a small trained adapter sits beside the block's own frozen weight.

attention.

Later the two streams are joined into one. The final layer projects back to 128 channels for the decoder.

The adapter is a small pair of matrices, A and B, trained beside each frozen weight. The adapter attaches inside the transformer blocks. It takes the attention projections and the feed-forward linears, and in the single-stream blocks the two fused linears. The block output adds a scaled product of A and B to the frozen term. Only A and B are trained, so the base weights never change. The adapter carries the head swap behaviour.

## 1.6 Generation

Generation (Figure 4) covers conditioning, sampling and decoding. A fixed prompt of 409 characters names Picture 1 as the body and Picture 2 as the head.

Qwen3-4B encodes the prompt once per session on the CPU, about 80 seconds, and its memory is then freed. Both pictures become VAE latents: the 512x512 body crop a 64x64x32 grid of about 1024 tokens, the 256x256 portrait about 256 tokens.

FLUX.2-klein-4B renders each frame in four sampling steps. Each step moves the latent along the direction the model predicts. The sampler calls the model once per step and feeds the result back. The text embeddings and the two image latents guide every step. A community-trained LoRA adapter of 80 adapted weights carries the head swap.

The same VAE decodes the final latent into pixels. Picture order is part of the adapter

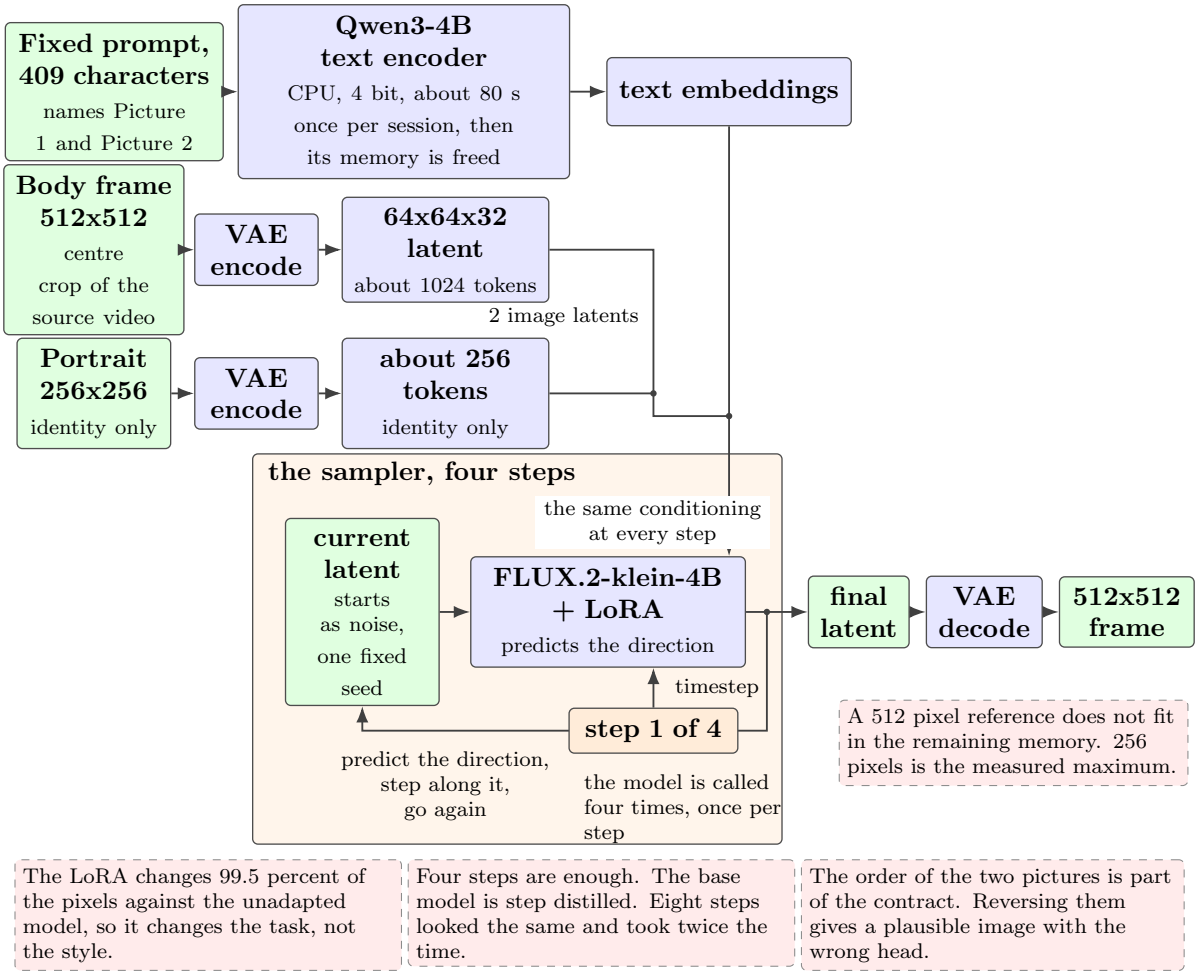



Figure 4: Inside the generation stage: Qwen3-4B encodes the prompt once, and the sampler then calls the model four times, once per step. Each call receives the current step number and the same conditioning, and the result is fed back into the loop. The last result is decoded into a 512x512 frame.

contract: the adapter's own documentation names Picture 1 the body and Picture 2 the head, and says not to reverse the images. A reversed order gives the wrong head.

The generator runs as a 4-bit GGUF file, and 256 pixels is the largest reference that fits. A detector refuses a frame with more than one face, and the gate has no bypass.

The output is 96 frames from 5.8 to 9.0 seconds of the source, all with one fixed seed. Each frame took 16.3 seconds, so the stage used 26.1 minutes.

### 1.6.1 Adapter version comparison

The recorded clip uses adapter v1 throughout. A newer adapter, v1.1, was released during the work, and we tested two prompt variants, A and B. Table 1 compares the two versions.

v1.1 ran, but it went far past the memory of the card, which is why it slowed down so much. The v1.1 adapter has rank 512 against the 128 of v1, so it is four times larger. That is why it went past the card and slowed down. The extra time was not worth it for us.

We judged the look by eye and preferred v1, so the clip kept the v1 output. No metric script was run on v1.1, so every other number here comes from v1. The grids in Figure 5 are the only evidence.

| | v1 (recorded) | v1.1 (new) |
|---|---|---|
| Adapter rank | 128 | 512 |
| Seconds per frame | 16.34 | about 199, twelve times slower |
| Peak VRAM allocated | 3.60 GiB | 4.68 GiB, past the 4.0 GiB of the card |
| Peak system RAM | 5.48 GiB | 5.50 GiB |
| Cost for 96 frames | 26 minutes | about 5.3 hours |

Table 1: Adapter v1 against v1.1 on the same machine; only v1 was measured with the metric script.

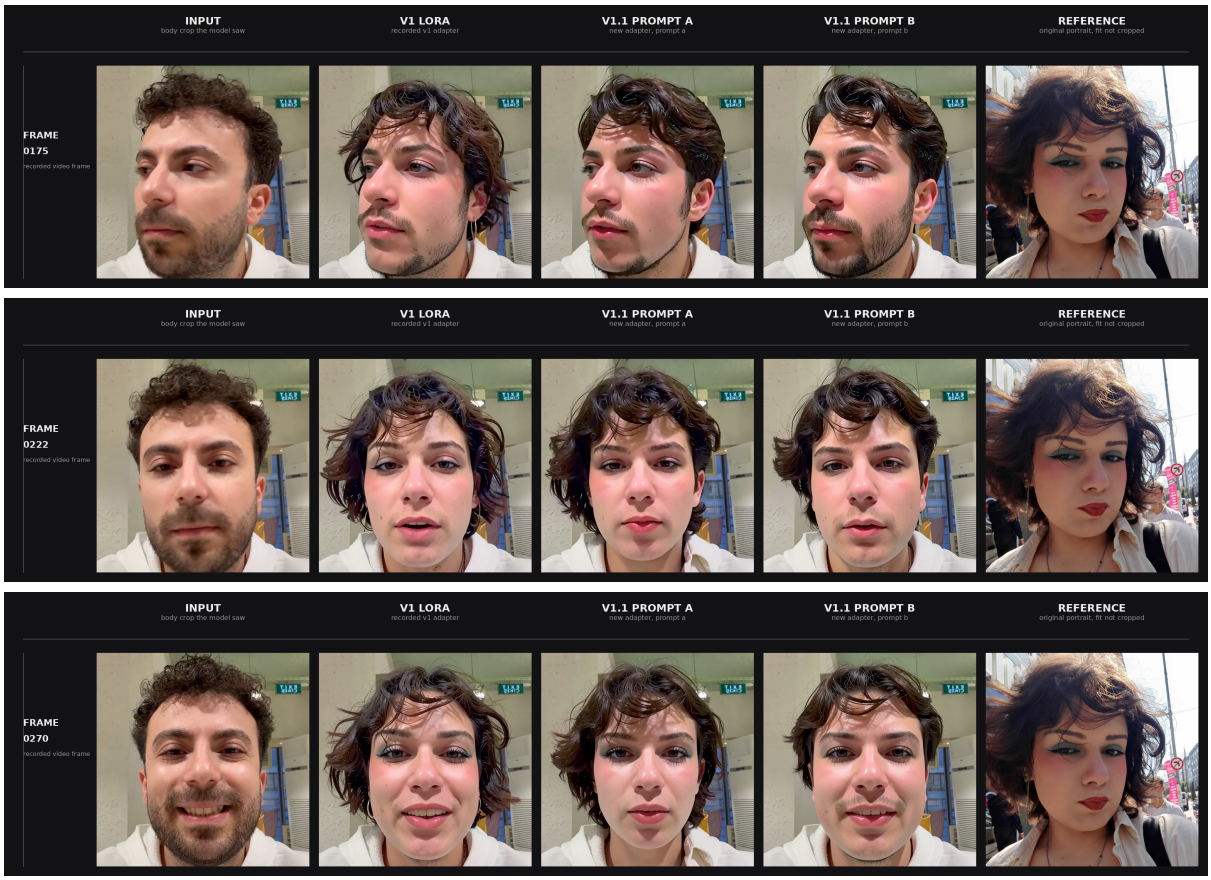



Figure 5: Each row is one source frame, the frame number printed inside. The five columns run from the input body crop through the v1 adapter, v1.1 with prompt A, v1.1 with prompt B, to the reference portrait; only the adapter version and the prompt variant change.

### 1.7 Face restoration

GFPGAN v1.4 cleans every frame (Figure 6). It detects and aligns the face, runs a restoration network, and composites the crop back. The weights are official, but we built the network in project code because the published wrappers do not import.

The stage costs 0.23 seconds per frame. The frames looked cleaner to us, and the mean face change came down, though the tail improved less.

Hair noise fell slightly, a small gain on its own. The fidelity weight does nothing, because the official inference path passes no weight argument. Restoration cannot remove flicker, because it sees one frame at a time.

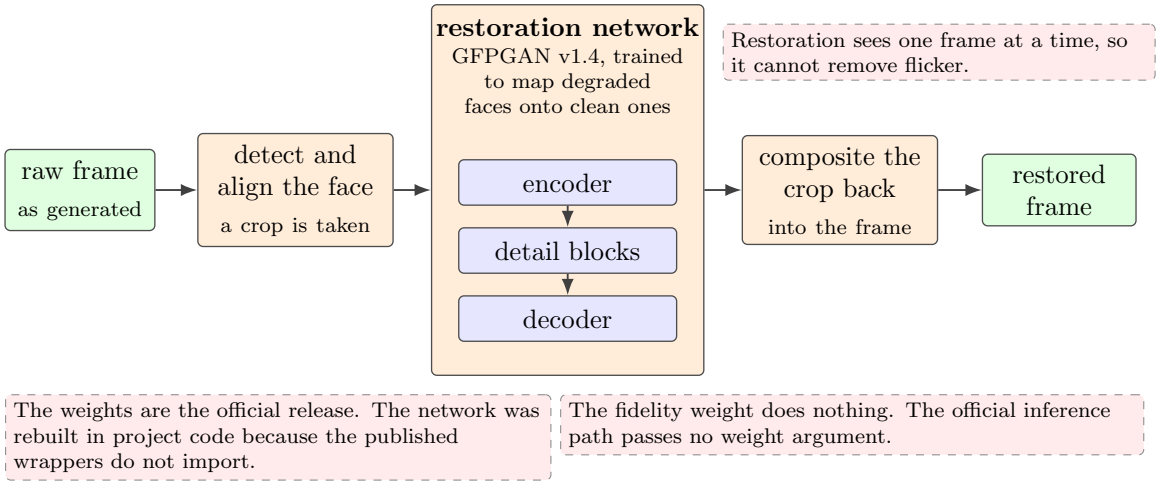



Figure 6: The face restoration stage. Each raw frame is taken on its own: the face is detected and aligned, a crop runs through the restoration network, and the restored crop is composited back into the frame.

| Frames smoothed | Strength | Face change | Distance from the control | p95 | Consecutive identity | Hair noise |
|---|---|---|---|---|---|---|
| Raw | 0.3 | 6.9599 | +0.3034 | 16.2944 | 0.9330 | 32.2358 |
| Raw | 0.6 | 6.6663 | +0.0098 | 15.6581 | 0.9391 | 32.0603 |
| Restored, then flow | 0.3 | 6.6303 | -0.0262 | 15.2802 | 0.9377 | 31.7306 |
| Restored, then flow | 0.6 | 6.3071 | -0.3494 | 14.7575 | 0.9431 | 31.5448 |

Table 2: The smoothing sweep: two frame sets at two strengths, all measured on the same 96 frames.

### 1.8 Temporal smoothing

The smoothing stage compares frames with each other and removes the boil (Figure 7). The per-pixel deflicker averages each pixel with its temporal neighbours, and it is too strong. It costs 0.007 seconds per frame.

The deflicker pushes the face change below the unswapped level, so it removes real motion as well as the boil. Optical flow estimates the dense motion between neighbouring frames and warps them onto the current one. Because it follows the same surface point across time, a moving head does not smear. It costs 0.036 seconds per frame.

The control fixes the real motion level at 6.66, and the distance column is the face change minus that level. At strength 0.3 the chained pass comes close enough to the control, and the raw pass stays above it. A stronger setting sits below the control, which is over-smoothing.

The recommended output is the chained set at strength 0.3. It beats the flow-only variant on face change, on the tail and on hair noise. Only consecutive identity is slightly lower. The deflicker reaches its low face change only by destroying motion, so we kept flow.

### 1.9 Reassembly

The encoder turns a frame set into a playable clip (Figure 8). It writes H.264 at 29.969 fps, muxes the original audio back, and trims both streams. Its output is a 512x512 clip of 3.203 seconds.

The encode adds nothing to the pixels. In the raw swapped set the face boils, and the results table holds the values.

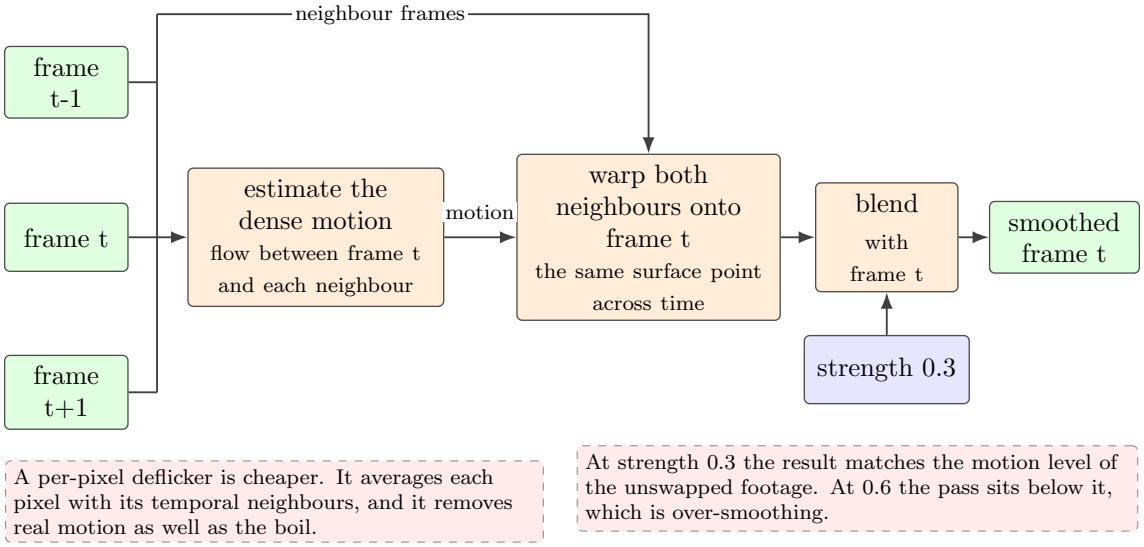



Figure 7: The temporal smoothing stage. Each frame is compared with its two neighbours: the dense motion is estimated, the neighbours are warped onto frame t and the three are blended at strength 0.3.

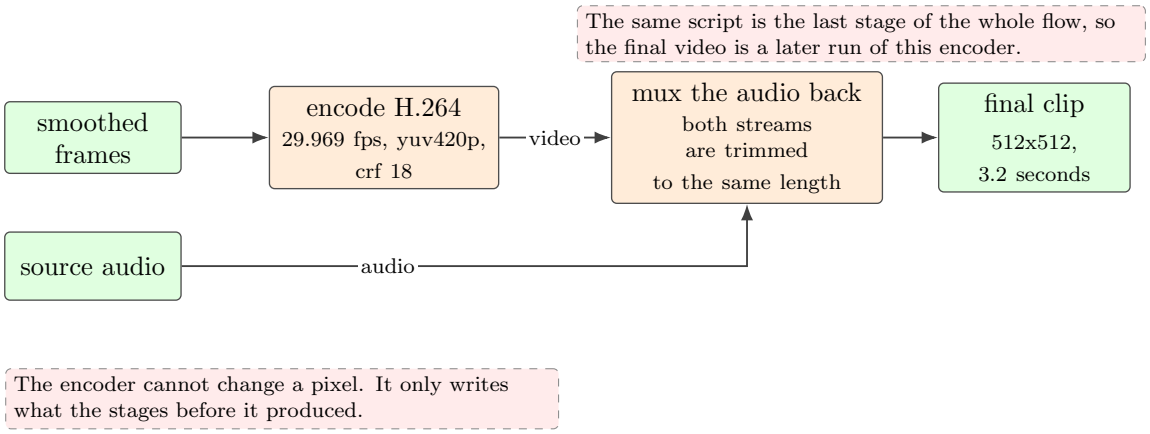



Figure 8: The reassembly stage. The smoothed frames are encoded as H.264 at the source rate and the original audio is muxed back; both streams are trimmed to the same length.

## 1.10 Results

All numbers come from one script on the same 96 frames. Lower hair noise is better, and higher identity values are better. Face change should sit near the control level, because a value below it means the smoother removed real motion. The control row is the unswapped source footage.

Identity to the portrait confirms the swap direction. The unswapped footage scores near zero, and every swapped variant scores positive. The recommended set held up well against the control on the mean values. Figure 9 shows the effect of each stage on one frame.

## 1.11 Problems and decisions

Attaching the adapter before the first forward pass cost 46 to 48 seconds per frame. Attaching it after one plain pass cost 16 seconds per frame. Peak VRAM was the same in both orders.

We standardised on the fast order. The internal cause was not established, but the effect reproduces exactly and explains an earlier timing discrepancy.

One fixed seed covers the whole clip, so the noise input is constant. The boil still happens,

| Stage | Face change (mean) | Face change (p95) | Consecutive identity | Identity to the portrait | Hair noise |
|---|---|---|---|---|---|
| Control (unswapped source footage) | 6.6565 | 12.3488 | 0.9852 | −0.0356 | 31.5999 |
| Raw head swap | 7.4510 | 17.3794 | 0.9228 | 0.2153 | 32.5703 |
| Restoration | 7.1602 | 16.2097 | 0.9273 | 0.2153 | 32.0721 |
| Flow on the raw frames | 6.6663 | 15.6581 | 0.9391 | 0.2148 | 32.0603 |
| Restoration then flow, strength 0.3 (recommended) | 6.6303 | 15.2802 | 0.9377 | 0.2152 | 31.7306 |
| Deflicker (alternative) | 5.6269 | 12.6650 | 0.9357 | 0.2106 | 31.8960 |

Table 3: All measured variants on the same 96 frames.



body input raw swap after restoration after smoothing

Figure 9: The same frame 0175 at four points of the flow; the four images are all 512x512.

because the model is sensitive to small input changes. Only a pass across frames reduces it.

Memory set the last limit; larger references do not fit. The largest reference we could use is 256 pixels.

## 1.12 Limits

The results table shows a mean at the control level, but the tail stays high, so the picture still jumped on some frames. Each step works inside a frame or a short window, so no step removed that tail. The 256-pixel reference maximum is the second limit on this card.